

# Proyecto de Inteligencia Artificial: Packing 3D con Algoritmo Genético

Abraham Másmela  
amasmela@unal.edu.co

Juan José Rojas  
jrojasal@unal.edu.co

*Universidad Nacional de Colombia*  
*2024-II*

2024-09-21

## 1. Introducción

La optimización de packing 3D o nesting 3D es un problema fundamental en la industria de la logística, manufactura y diseño, donde se busca maximizar el uso del espacio disponible para ubicar objetos tridimensionales dentro de un contenedor determinado. Este tipo de problemas pertenece a la clase de problemas NP-completos, lo que significa que no existe un algoritmo eficiente que resuelva todos los casos en tiempo polinomial. Sin embargo, mediante el uso de métodos heurísticos y algoritmos aproximados, es posible encontrar soluciones aceptables en un tiempo razonable.

El packing 3D es utilizado en diversas aplicaciones prácticas, como el embalaje de productos para su transporte, el diseño de patrones para cortar materiales (madera, metal o telas), y en la impresión 3D, donde es necesario organizar múltiples piezas dentro de un volumen para su fabricación eficiente.

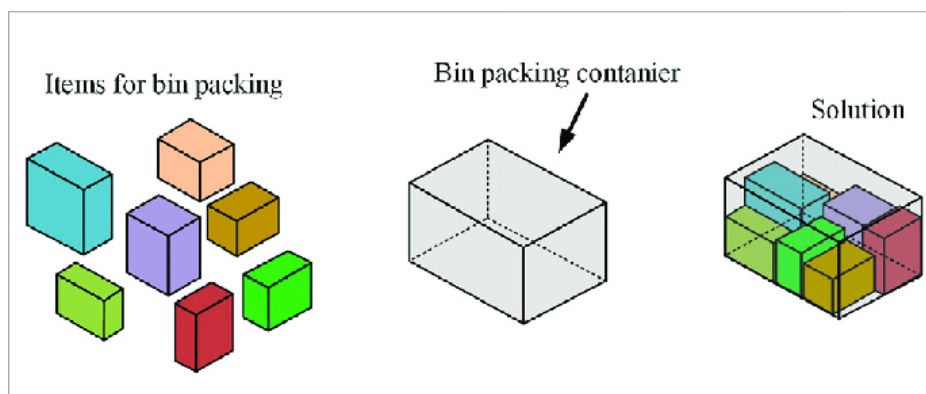


Figura 1: Ejemplo de Empaquetado 3D

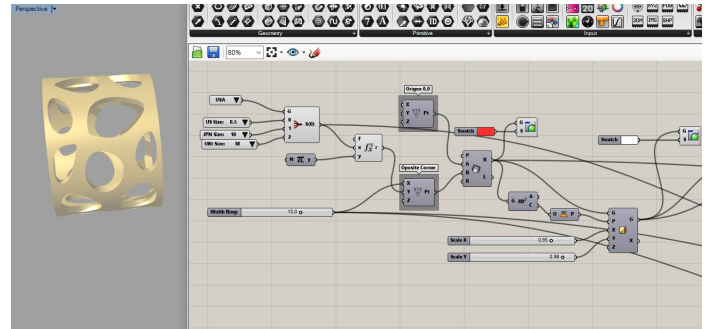
Este tipo de problema, conocido por su complejidad, ha motivado el desarrollo de diversas técnicas de optimización, siendo los **algoritmos genéticos** una de las más efectivas para abordar su naturaleza combinatoria.

En este contexto, se implementa el uso de **algoritmos genéticos** dentro del entorno de diseño asistido por

computadora **CAD Rhino 3D** y su complemento paramétrico **Grasshopper**, una plataforma potente para el modelado y la optimización geométrica. Grasshopper permite definir algoritmos visualmente, lo que facilita la integración de procesos evolutivos como los algoritmos genéticos, que simulan mecanismos de selección natural para buscar soluciones óptimas. A través de la generación de poblaciones de soluciones, su evaluación mediante funciones de aptitud y la aplicación de operadores como cruza y mutación, es posible encontrar configuraciones eficientes que maximicen el uso del espacio en problemas de packing 3D.



(a) Rhino 3D CAD



(b) Ejemplo de programación en Grasshopper

Figura 2

## 2. Desarrollo

### 2.1. Generación de los cromosomas y la población

Se requiere organizar un conjunto de cajas dentro de un contenedor, empleando un algoritmo genético. Para desarrollar este algoritmo, primero es necesario definir la estructura del **cromosoma o individuo** y los genes que lo componen, es decir, la información que representa la solución. En este caso, un cromosoma corresponde a una posible disposición de las cajas dentro del contenedor, donde los genes están formados por las posiciones de los centroides de cada caja en relación con el contenedor.

Para generar estos cromosomas, se ha implementado un código en Python que organiza las cajas aleatoriamente dentro del contenedor, garantizando que no se superpongan ni excedan los límites del contenedor. A partir de este proceso de organización, se pueden generar tantos cromosomas como sea necesario. En este caso particular, se ha definido una población inicial de 14 individuos, cada uno representando una disposición diferente de las cajas dentro del contenedor.

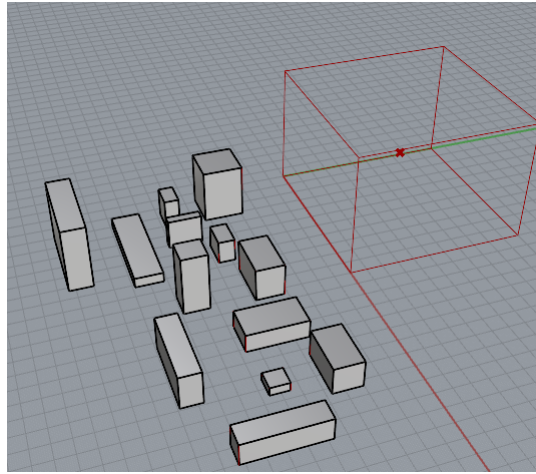


Figura 3: Cajas a organizar y contenedor

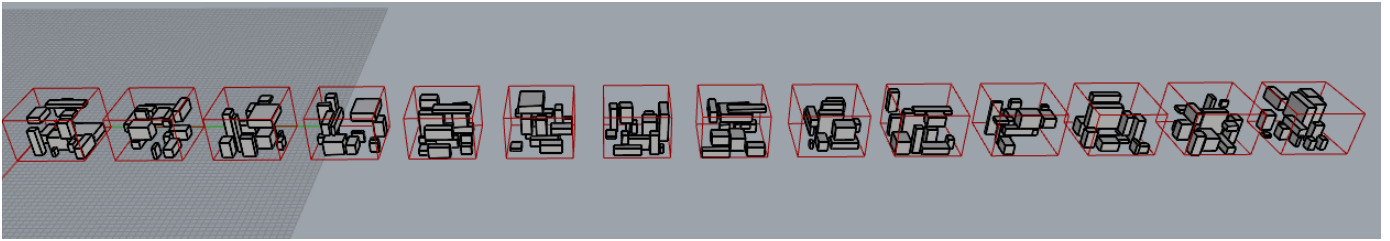


Figura 4: Población de 14 formas de organización de las cajas en el contenedor

## 2.2. Función de aptitud y probabilidad de selección

En este caso, la función de aptitud se basa en el volumen de la **bounding box**, que es la caja más pequeña posible que puede encerrar un grupo de elementos. Para cada disposición de las cajas organizadas, se calcula la bounding box correspondiente, y se determina su volumen. La disposición que resulte en la bounding box con el volumen más pequeño se considera la mejor, ya que implica que las cajas están organizadas de manera más compacta, ocupando el menor espacio posible dentro del contenedor.

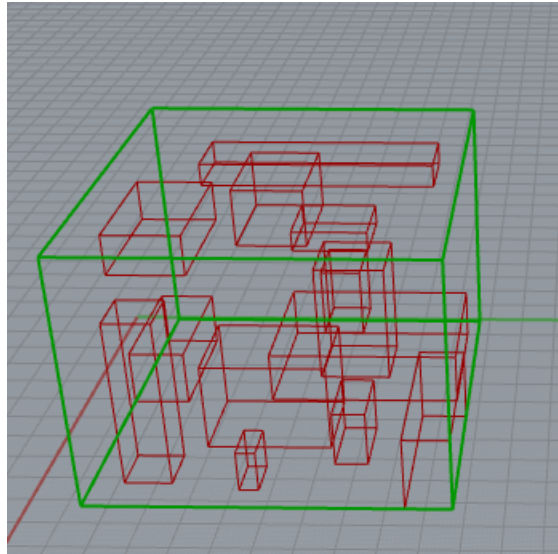


Figura 5: Ejemplo de organización de cajas en su bounding box

La función de aptitud queda como:

$$Fa = 100 - \frac{Volumen_{BoundingBox}}{Volumen_{Contenedor}} * 100$$

De esta forma, mientras menor sea el volumen de la bounding box mayor será el número de aptitud.

La probabilidad de selección de cada caja se determina en función de su propia función de aptitud y de la aptitud total de la población, que se calcula como la suma de las funciones de aptitud de todas las cajas. De este modo, las cajas con mejor aptitud tienen una mayor probabilidad de ser seleccionadas, ya que contribuyen de manera más eficiente al objetivo general de optimización.

A continuación, se presentan los valores del volumen de la bounding box y la probabilidad de selección correspondientes a la primera iteración. Posteriormente, se lleva a cabo una selección por ruleta, donde las disposiciones de cajas con mayor probabilidad de ser seleccionadas, según su aptitud, son elegidas para continuar en el proceso evolutivo.

|    | {0}       |
|----|-----------|
| 0  | 6.5477e+8 |
| 1  | 6.4486e+8 |
| 2  | 7.2274e+8 |
| 3  | 7.296e+8  |
| 4  | 6.8782e+8 |
| 5  | 6.2242e+8 |
| 6  | 7.4184e+8 |
| 7  | 7.6569e+8 |
| 8  | 4.4753e+8 |
| 9  | 7.2797e+8 |
| 10 | 6.973e+8  |
| 11 | 7.1605e+8 |
| 12 | 6.2649e+8 |
| 13 | 8.3572e+8 |

(a) Volumen de la bounding box de cada organización de cajas de la primera iteración en mm<sup>3</sup>

|    | {0;0}    |
|----|----------|
| 0  | 8.139534 |
| 1  | 8.44422  |
| 2  | 6.050417 |
| 3  | 5.839698 |
| 4  | 7.123947 |
| 5  | 9.133775 |
| 6  | 5.463374 |
| 7  | 4.730546 |
| 8  | 14.50929 |
| 9  | 5.889905 |
| 10 | 6.832458 |
| 11 | 6.256191 |
| 12 | 9.008711 |
| 13 | 2.577934 |

(b) Probabilidad de selección de cada organización de cajas en % de la primera iteración

Figura 6

### 2.3. Cruce de los cromosomas seleccionados y repetición del algoritmo

Durante el proceso de cruce, se seleccionan dos cromosomas, es decir, dos disposiciones de cajas organizadas en sus respectivos contenedores. A partir de estos, se elige aleatoriamente una caja de cada contenedor, intercambiando sus posiciones. Esto da lugar a la generación de dos nuevos individuos, o "hijos". En caso de que este intercambio provoque una colisión con otras cajas, se selecciona una nueva posición aleatoria para la caja hasta encontrar una ubicación válida que evite las colisiones.

Posteriormente, se repite el algoritmo evaluando la función de aptitud de los nuevos individuos generados (los hijos). A continuación, se lleva a cabo nuevamente el proceso de selección y cruce, permitiendo que las mejores disposiciones continúen evolucionando y mejorando en cada iteración. Este ciclo se repite hasta alcanzar un criterio de convergencia o un número predefinido de generaciones.

A continuación se presenta el esquema general de la programación en Grasshopper, que incluye los procesos de generación de la población, evaluación de la función de aptitud, selección y mutación.

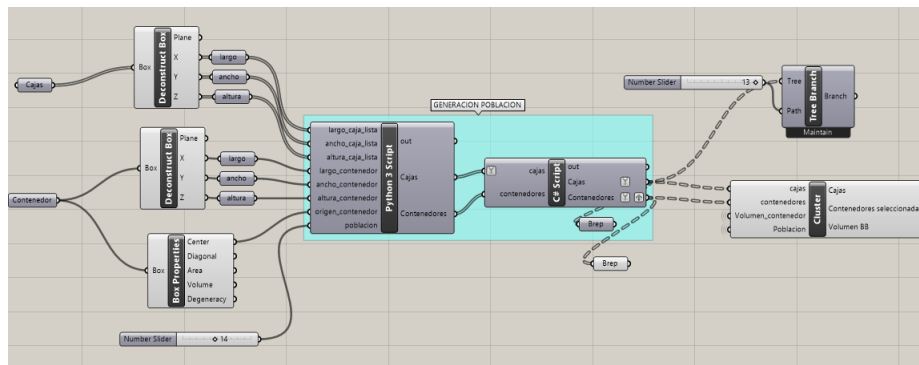


Figura 7: Programación en grasshopper de la generación de la población

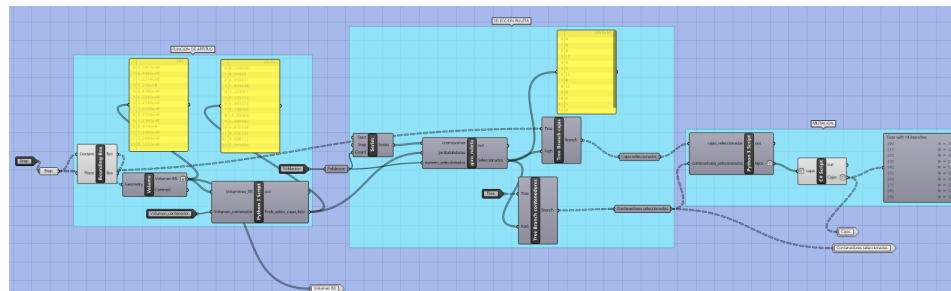


Figura 8: Programación en grasshopper de la función de aptitud, selección y mutación

### 3. Resultados

Se implementó el algoritmo genético para organizar 14 cajas, utilizando una población inicial de 14 disposiciones. Después de 30 iteraciones, el algoritmo convergió hacia una organización estable que dejó de evolucionar, alcanzando un punto de estancamiento. En la figura 9, se presentan los volúmenes de la bounding box correspondientes a la primera iteración y a la iteración 30. Cabe recordar que, cuanto menor sea el volumen de la bounding box, mejor es la organización de las cajas.

|    | {0}       |
|----|-----------|
| 0  | 6.5477e+8 |
| 1  | 6.4486e+8 |
| 2  | 7.2274e+8 |
| 3  | 7.296e+8  |
| 4  | 6.8782e+8 |
| 5  | 6.2242e+8 |
| 6  | 7.4184e+8 |
| 7  | 7.6569e+8 |
| 8  | 4.4753e+8 |
| 9  | 7.2797e+8 |
| 10 | 6.973e+8  |
| 11 | 7.1605e+8 |
| 12 | 6.2649e+8 |
| 13 | 8.3572e+8 |

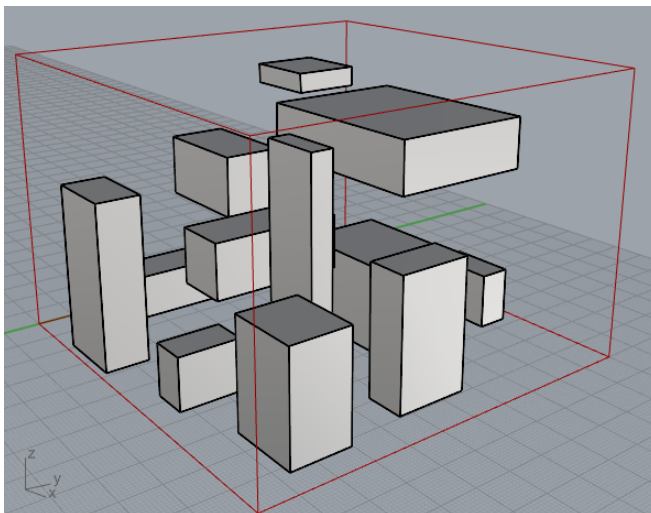
(a) Volumen del bounding box de las cajas organizadas en la primera iteración

|    | {0}       |
|----|-----------|
| 0  | 4.8744e+8 |
| 1  | 4.8744e+8 |
| 2  | 4.8744e+8 |
| 3  | 4.8744e+8 |
| 4  | 4.8744e+8 |
| 5  | 4.8744e+8 |
| 6  | 4.8744e+8 |
| 7  | 4.8744e+8 |
| 8  | 4.8744e+8 |
| 9  | 4.8744e+8 |
| 10 | 4.8744e+8 |
| 11 | 4.8744e+8 |
| 12 | 4.8744e+8 |
| 13 | 4.8744e+8 |

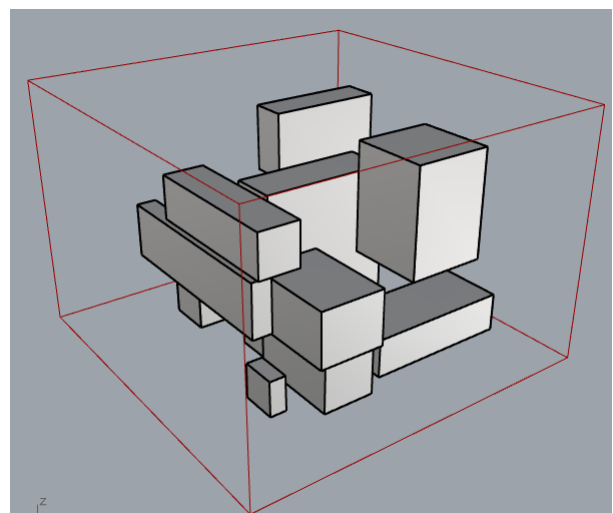
(b) Volumen del bounding box de las cajas organizadas en la 30 iteración

Figura 9: Comparación de volúmenes de la bounding box en diferentes iteraciones

Como se aprecia en la figura anterior, en la iteración 30 se observó una reducción en el volumen de la bounding box, lo que indica que el algoritmo genético fue efectivo hasta cierto punto. En la figura 10, se comparan las disposiciones de las cajas en la primera iteración y en la iteración 30, evidenciando la mejora en la organización.



(a) Cajas organizadas en la primera iteración



(b) Cajas organizadas en la 30 iteración

Figura 10: Comparación de cajas organizadas en diferentes organizaciones

Como se observa en la figura, en la iteración 30 las cajas están dispuestas de manera más compacta,

lo que indica que el algoritmo genético ha sido efectivo hasta cierto punto. No obstante, persisten algunos espacios vacíos y las cajas no están sujetas a la gravedad, lo que evidencia que la solución no es completamente óptima. A partir de la iteración 30, la disposición de las cajas se estabiliza y no cambia, lo que sugiere que el algoritmo ha alcanzado un punto de estancamiento. A pesar de estas limitaciones, el resultado obtenido representa un buen acercamiento al objetivo de optimización.

## 4. Conclusiones

- El algoritmo genético implementado ha sido exitoso hasta cierto punto, logrando una disposición más compacta de las cajas tras varias iteraciones. Sin embargo, el proceso alcanzó un punto de estancamiento, donde las soluciones dejaron de mejorar. A pesar de estas limitaciones, el enfoque representa un buen avance en la optimización del packing 3D.
- Uno de los principales aspectos a mejorar es incorporar la acción de la gravedad en el algoritmo. Actualmente, las cajas no se ven afectadas por esta fuerza, lo que provoca que algunas queden flotando, afectando la precisión de la solución. Implementar la gravedad permitiría obtener una organización más realista y eficiente.
- Los resultados del algoritmo podrían mejorarse aumentando el tamaño de la población inicial. Una mayor diversidad en las soluciones permitiría explorar un espacio de búsqueda más amplio, incrementando la probabilidad de encontrar configuraciones óptimas. Sin embargo, este enfoque requeriría un mayor poder computacional debido al incremento en el número de individuos a procesar.
- El uso de Rhino 3D junto con Grasshopper ha sido fundamental para implementar y visualizar el algoritmo genético de packing 3D. Grasshopper permitió la generación automática de poblaciones y la manipulación geométrica de las cajas, facilitando tanto la experimentación como el análisis visual de las soluciones. Esta integración demuestra el potencial de estas herramientas para resolver problemas complejos de optimización espacial de manera eficiente y visualmente intuitiva.

## 5. Código utilizado

### 5.1. Generación de población inicial

```
1 import rhinoscriptsyntax as rs
2 import Rhino.Geometry as rg
3 import random
4 import math
5
6 # Funcion que ajusta la altura de las cajas para que caigan al suelo o sobre
   otra caja
7 def ajustar_altura_cajas(cajas, altura_contenedor):
8     for caja in cajas:
9         caja_bbox = caja.ToBrep().GetBoundingBox(True)
10        altura_inicial = caja_bbox.Min.Z
11        altura_final = 0 # Altura del suelo
12
```

```

13     while True:
14         # Verificar si hay colision con alguna caja debajo de la actual
15         colision = False
16         for otra_caja in cajas:
17             if caja != otra_caja:
18                 otra_caja_bbox = otra_caja.ToBrep().GetBoundingBox(True)
19                 if (se_solapan_en_x_y(caja_bbox, otra_caja_bbox) and
20                     otra_caja_bbox.Max.Z <= caja_bbox.Min.Z):
21                     altura_final = otra_caja_bbox.Max.Z # Toca la caja
22                     debajo
23                     colision = True
24                     break
25
26         if colision or altura_inicial <= 0:
27             break # Caja en el suelo o sobre otra caja
28
29         # Mover la caja hacia abajo
30         caja.Transform(rg.Transform.Translation(0, 0, -1))
31         altura_inicial -= 1
32
33 # Funcion para verificar si dos BoundingBoxes se solapan en el plano XY
34 def se_solapan_en_x_y(bbox1, bbox2):
35     return not (bbox1.Max.X < bbox2.Min.X or bbox1.Min.X > bbox2.Max.X or
36                 bbox1.Max.Y < bbox2.Min.Y or bbox1.Min.Y > bbox2.Max.Y)
37
38 # Funcion que genera cajas aleatorias dentro de un contenedor
39 def generar_contenedor_y_cajas(origen_contenedor, largo_contenedor,
40     ancho_contenedor, altura_contenedor, largo_caja_lista, ancho_caja_lista,
41     altura_caja_lista):
42
43     # Coordenadas origen contenedor
44     contenedor_centro_x = origen_contenedor.X
45     contenedor_centro_y = origen_contenedor.Y
46     contenedor_centro_z = origen_contenedor.Z
47
48     # Funcion para verificar si dos BoundingBoxes se solapan
49     def verificar_colision(caja1, caja2):
50         bbox1 = caja1.ToBrep().GetBoundingBox(True)
51         bbox2 = caja2.ToBrep().GetBoundingBox(True)
52
53         no_colision_x = bbox1.Max.X < bbox2.Min.X or bbox1.Min.X > bbox2.Max
54         .X
55         no_colision_y = bbox1.Max.Y < bbox2.Min.Y or bbox1.Min.Y > bbox2.Max
56         .Y
57         no_colision_z = bbox1.Max.Z < bbox2.Min.Z or bbox1.Min.Z > bbox2.Max
58         .Z
59
60         return not (no_colision_x or no_colision_y or no_colision_z)

```

```

56 # Funcion para verificar si la caja esta completamente dentro del
    contenedor
57 def verificar_dentro_contenedor(caja, contenedor_centro_x,
    contenedor_centro_y, contenedor_centro_z, largo_contenedor,
    ancho_contenedor, altura_contenedor):
58     esquinas = caja.GetCorners()
59
60     for esquina in esquinas:
61         if not (contenedor_centro_x - largo_contenedor / 2 <= esquina.X
62                 <= contenedor_centro_x + largo_contenedor / 2 and
63                 contenedor_centro_y - ancho_contenedor / 2 <= esquina.Y
64                 <= contenedor_centro_y + ancho_contenedor / 2 and
65                 contenedor_centro_z - altura_contenedor / 2 <= esquina.Z
66                 <= contenedor_centro_z + altura_contenedor / 2):
67             return False
68
69     return True
70
71 # Funcion para generar una caja aleatoria sin colisiones dentro del
    contenedor
72 def generar_caja_aleatoria_sin_colision(cajas_existentes,
73     contenedor_centro_x, contenedor_centro_y, contenedor_centro_z,
74     largo_contenedor, ancho_contenedor, altura_contenedor, largo_caja,
75     ancho_caja, altura_caja):
76     intentos = 0
77     max_intentos = 100
78
79     while intentos < max_intentos:
80         x = random.uniform(contenedor_centro_x - largo_contenedor / 2 +
81                             largo_caja / 2, contenedor_centro_x + largo_contenedor / 2 -
82                             largo_caja / 2)
83         y = random.uniform(contenedor_centro_y - ancho_contenedor / 2 +
84                             ancho_caja / 2, contenedor_centro_y + ancho_contenedor / 2 -
85                             ancho_caja / 2)
86         z = random.uniform(contenedor_centro_z - altura_contenedor / 2 +
87                             altura_caja / 2, contenedor_centro_z + altura_contenedor / 2
88                             - altura_caja / 2)
89
90         punto_origen = rg.Point3d(x, y, z)
91         caja = rg.Box(rg.Plane(punto_origen, rg.Vector3d.ZAxis), rg.
92             Interval(0, largo_caja), rg.Interval(0, ancho_caja), rg.
93             Interval(0, altura_caja))
94
95         rot_x = random.choice([0, 90, 180, 270])
96         rot_y = random.choice([0, 90, 180, 270])
97         rot_z = random.choice([0, 90, 180, 270])
98
99         if rot_x != 0:
100             rotacion_x = rg.Transform.Rotation(math.radians(rot_x), rg.
101                 Vector3d.XAxis, punto_origen)

```

```

87         caja.Transform(rotacion_x)
88
89     if rot_y != 0:
90         rotacion_y = rg.Transform.Rotation(math.radians(rot_y), rg.
91             Vector3d.YAxis, punto_origen)
92         caja.Transform(rotacion_y)
93
94     if rot_z != 0:
95         rotacion_z = rg.Transform.Rotation(math.radians(rot_z), rg.
96             Vector3d.ZAxis, punto_origen)
97         caja.Transform(rotacion_z)
98
99     if not verificar_dentro_contenedor(caja, contenedor_centro_x,
100         contenedor_centro_y, contenedor_centro_z, largo_contenedor,
101         ancho_contenedor, altura_contenedor):
102         intentos += 1
103         continue
104
105     colision = False
106     for caja_existente in cajas_existentes:
107         if verificar_colision(caja, caja_existente):
108             colision = True
109             break
110
111     if not colision:
112         return caja
113
114     intentos += 1
115
116     return None
117
118 # Generar el contenedor centrado en el origen
119 contenedor = rg.Box(rg.Plane(origen_contenedor, rg.Vector3d.ZAxis),
120     rg.Interval(-largo_contenedor / 2, largo_contenedor
121         / 2),
122     rg.Interval(-ancho_contenedor / 2, ancho_contenedor
123         / 2),
124     rg.Interval(-altura_contenedor / 2,
125         altura_contenedor / 2))
126
127 # Lista para almacenar las cajas generadas
128 Cajas = []
129
130 # Generar todas las cajas
131 for largo_caja, ancho_caja, altura_caja in zip(largo_caja_lista,
132     ancho_caja_lista, altura_caja_lista):
133     caja = generar_caja_aleatoria_sin_colision(Cajas,
134         contenedor_centro_x, contenedor_centro_y, contenedor_centro_z,
135         largo_contenedor,
136         ancho_contenedor,

```

```

127         altura_contenedor,
128         largo_caja, ancho_caja,
129         altura_caja)
130
131     if caja is not None:
132         Cajas.append(caja)
133     else:
134         print(f"No se pudo generar una caja sin colisiones para las
135             dimensiones: {largo_caja}, {ancho_caja}, {altura_caja}")
136
137     # Ajustar las alturas de las cajas
138     ajustar_altura_cajas(Cajas, altura_contenedor)
139
140     # Retornar las cajas y el contenedor
141     return Cajas, contenedor
142
143 # Lista para almacenar las cajas y contenedores generados
144 contenedores_y_cajas = []
145 Cajas=[]
146 Contenedores=[]
147
148 # Generar 10 contenedores y sus respectivas cajas
149 for i in range(poblacion):
150     # Mover el origen del contenedor hacia arriba en Y para cada iteraci n
151     origen_contenedor_nuevo = rg.Point3d(origen_contenedor.X,
152         origen_contenedor.Y + (ancho_contenedor + 500) * i, origen_contenedor
153         .Z)
154
155     # Generar el contenedor y sus cajas
156     cajas, contenedor = generar_contenedor_y_cajas(origen_contenedor_nuevo,
157         largo_contenedor, ancho_contenedor, altura_contenedor,
158         largo_caja_lista, ancho_caja_lista, altura_caja_lista)
159
160     # Guardar en la lista
161     contenedores_y_cajas.append((cajas, contenedor))
162     Cajas.append(cajas)
163     Contenedores.append(contenedor)

```

## 5.2. Función de aptitud

```

1
2 prob_selecc_cajas_lista=[]
3 aptitudes=[]
4
5
6
7 Suma_volumenes = sum(Volumenes_BB) #aptitud de toda la poblacion

```

```

8
9 #loop para sacar la aptitud de cada cromosoma
10 for volumen in Volumenes_BB:
11     aptitud=100-volumen/Volumen_contenedor*100
12     aptitudes.append(aptitud)
13
14
15 suma_aptitudes_poblacion=sum(aptitudes)
16
17 for aptitud in aptitudes:
18     prob_selecc_cajas=aptitud/suma_aptitudes_poblacion*100
19     prob_selecc_cajas_lista.append(prob_selecc_cajas)
20
21
22 Prob_selecc_cajas_lista=prob_selecc_cajas_lista

```

### 5.3. Giro de la ruleta para seleccion

```

1 import random
2 import numpy as np
3
4 def seleccion_por_ruleta(cromosomas, probabilidades, n_selecciones):
5
6     # Verificar que las probabilidades suman 1
7     #assert sum(probabilidades) == 1, "Las probabilidades deben sumar 1"
8
9     # Lista para almacenar los cromosomas seleccionados
10    seleccionados = []
11
12    # Seleccion de cromosomas
13    for _ in range(n_selecciones):
14        # Girar la ruleta (usamos random.choices con las probabilidades
15        # dadas)
16        seleccionado = random.choices(cromosomas, weights=probabilidades, k
17        =1)[0]
18        seleccionados.append(seleccionado)
19
20    return seleccionados
21
22
23 Seleccionados = seleccion_por_ruleta(cromosomas, probabilidades,
    numero_seleccionados)

```

### 5.4. Cruce de cajas

```

1 import rhinoscriptsyntax as rs
2 import Rhino.Geometry as rg
3 from Grasshopper import DataTree
4 from Grasshopper.Kernel.Data import GH_Path
5 import random
6 import math
7
8
9
10 def verificar_colision(caja1, caja2):
11     bbox1 = caja1.ToBrep().GetBoundingBox(True)
12     bbox2 = caja2.ToBrep().GetBoundingBox(True)
13
14     no_colision_x = bbox1.Max.X < bbox2.Min.X or bbox1.Min.X > bbox2.Max.X
15     no_colision_y = bbox1.Max.Y < bbox2.Min.Y or bbox1.Min.Y > bbox2.Max.Y
16     no_colision_z = bbox1.Max.Z < bbox2.Min.Z or bbox1.Min.Z > bbox2.Max.Z
17
18     return not (no_colision_x or no_colision_y or no_colision_z)
19
20 # Obtener las posiciones de los centros de las cajas y contenedores
21 def obtener_centro(bbox):
22     # Devuelve el centro de un BoundingBox
23     return bbox.Center
24
25 # Funci n para calcular el volumen de una caja
26 def calcular_volumen_caja(caja):
27     return caja.Volume
28
29 def data_tree_a_lista(data_tree):
30     lista = []
31     for path in data_tree.Paths:
32         rama = data_tree.Branch(path)
33         lista.extend(rama) # Agregar todos los elementos de la rama a la
                               lista
34     return lista
35
36 def eliminar_duplicado_por_volumen(cajas):
37     vol_menes_vistos = {}
38     cajas_a_eliminar = []
39
40     # Recorre todas las cajas
41     for caja in cajas:
42         volumen = calcular_volumen_caja(caja)
43
44         if volumen in vol_menes_vistos:
45             # Marca la caja actual para eliminarla
46             cajas_a_eliminar.append(caja)
47         else:
48             # Guarda la caja en el diccionario de vol menes vistos

```

```

49         vol_menes_vistos[volumen] = caja
50
51     # Elimina una caja por cada volumen duplicado encontrado
52     for caja in cajas_a_eliminar:
53         cajas.remove(caja)
54
55     return cajas
56
57     #####3
58 Hijos_corregidos_total=[]
59 def cruces_por_pares(cajas_seleccionadas, contenedores_seleccionados):
60     Hijos_corregidos =[]
61     # Supongamos que 'cajas_seleccionadas' y 'contenedores_seleccionados'
        son DataTree
62     # Vamos a crear dos nuevos DataTree para los hijos
63
64     for i in range(0, cajas_seleccionadas.BranchCount - 1, 2):
65         hijo_0 = DataTree[rg.Box]()
66         hijo_1 = DataTree[rg.Box]()
67         grupo_cajas_0 = cajas_seleccionadas.Branch(i) # Grupo i
68         grupo_cajas_1 = cajas_seleccionadas.Branch(i+1) # Grupo i+1
69         contenedor0 = contenedores_seleccionados.Branch(i)[0] # Contenedor
            del Grupo i
70         contenedor1 = contenedores_seleccionados.Branch(i+1)[0]
71
72         # Seleccionar una caja aleatoria del grupo 0
73         indice_aleatorio = random.randint(0, len(grupo_cajas_0) - 1)
74         caja_aleatoria_grupo_0 = grupo_cajas_0[indice_aleatorio]
75
76         # Calcular el volumen de la caja seleccionada
77         volumen_caja_0 = calcular_volumen_caja(caja_aleatoria_grupo_0)
78
79         # Margen de tolerancia para la comparaci n
80         tolerancia = 0.0001
81
82         # Buscar una caja en el grupo 1 que tenga un volumen equivalente
            dentro del margen de tolerancia
83         caja_equivalente_grupo_1 = None
84         indice_caja_1 = None
85         for i, caja in enumerate(grupo_cajas_1):
86             if abs(calcular_volumen_caja(caja) - volumen_caja_0) <
                tolerancia:
87                 caja_equivalente_grupo_1 = caja
88                 indice_caja_1 = i
89                 break
90
91         # Comprobar si se encontr una caja equivalente en el grupo 1
92         if caja_equivalente_grupo_1 is not None:
93             print(f"Caja_seleccionada_de_Grupo_0:_{caja_aleatoria_grupo_0}")

```

```

94     print(f"Caja_equivalente_encontrada_en_grupo_1:{\n{
95         caja_equivalente_grupo_1}")
96
97     # Obtener los BoundingBoxes de las cajas y contenedores
98     bbox_caja_0 = caja_aleatoria_grupo_0.BoundingBox
99     bbox_caja_1 = caja_equivalente_grupo_1.BoundingBox
100     bbox_contenedor_0 = contenedor0.BoundingBox
101     bbox_contenedor_1 = contenedor1.BoundingBox
102
103     # Obtener los centros de las cajas y contenedores
104     centro_caja_0 = obtener_centro(bbox_caja_0)
105     centro_caja_1 = obtener_centro(bbox_caja_1)
106     centro_contenedor_0 = obtener_centro(bbox_contenedor_0)
107     centro_contenedor_1 = obtener_centro(bbox_contenedor_1)
108
109     # Calcular las posiciones relativas de las cajas en sus
110     respectivos contenedores
111     posicion_relativa_0 = rg.Vector3d(centro_caja_0) - rg.Vector3d(
112         centro_contenedor_0)
113     posicion_relativa_1 = rg.Vector3d(centro_caja_1) - rg.Vector3d(
114         centro_contenedor_1)
115
116     # Crear nuevas instancias de las cajas (copiar)
117     caja_nueva_0 = rg.Box(caja_aleatoria_grupo_0.Plane,
118         caja_aleatoria_grupo_0.BoundingBox)
119     caja_nueva_1 = rg.Box(caja_equivalente_grupo_1.Plane,
120         caja_equivalente_grupo_1.BoundingBox)
121
122     # Ahora aplicamos las posiciones relativas de las cajas cruzadas
123     a los contenedores opuestos
124     nueva_posicion_caja_0 = rg.Point3d(centro_contenedor_1) +
125         posicion_relativa_0 # Caja 0 dentro del contenedor 1
126     nueva_posicion_caja_1 = rg.Point3d(centro_contenedor_0) +
127         posicion_relativa_1 # Caja 1 dentro del contenedor 0
128
129     # Aplicar las nuevas transformaciones de traslaci n para mover
130     las cajas a sus nuevas posiciones
131     transformacion_0 = rg.Transform.Translation(
132         nueva_posicion_caja_0 - centro_caja_0)
133     transformacion_1 = rg.Transform.Translation(
134         nueva_posicion_caja_1 - centro_caja_1)
135
136     caja_nueva_0.Transform(transformacion_0)
137     caja_nueva_1.Transform(transformacion_1)
138
139     # Crear los hijos con las cajas intercambiadas en sus posiciones
140     relativas
141     for i, caja in enumerate(grupo_cajas_0):
142         if i == indice_aleatorio:

```

```

130         hijo_0.Add(caja_nueva_1, GH_Path(0)) # Aadir la caja
        transformada del grupo 1
131     else:
132         hijo_0.Add(caja, GH_Path(0)) # Aadir las dem s cajas
        del grupo 0
133
134     for i, caja in enumerate(grupo_cajas_1):
135         if i == indice_caja_1:
136             hijo_1.Add(caja_nueva_0, GH_Path(0)) # Aadir la caja
        transformada del grupo 0
137         else:
138             hijo_1.Add(caja, GH_Path(0)) # Aadir las dem s cajas
        del grupo 1
139     else:
140         print("No se encontr una caja equivalente en el Grupo 1.")
141
142     #-----
143     # Funci n para verificar si dos BoundingBoxes se solapan
144     colision0 = False
145     #verificar_colision(caja, caja_existente):
146     # Iterar sobre todas las ramas del DataTree
147     for path in hijo_0.Paths:
148         # Obtener los elementos de la rama actual
149         rama = hijo_0.Branch(path)
150         # Iterar sobre los elementos de la rama
151         for caja in rama:
152             # Aqu puedes trabajar con cada 'caja'
153             if verificar_colision(caja_nueva_1, caja):
154                 if(calcular_volumen_caja(caja_nueva_1)==
        calcular_volumen_caja(caja)):
155                     colision0=False
156             else:
157                 colision0 = True
158                 break
159     print("colision en hijo_0:")
160     print(colision0)
161
162     colision1 = False
163     #verificar_colision(caja, caja_existente):
164     # Iterar sobre todas las ramas del DataTree
165     for path in hijo_1.Paths:
166         # Obtener los elementos de la rama actual
167         rama = hijo_1.Branch(path)
168         # Iterar sobre los elementos de la rama
169         for caja in rama:
170             # Aqu puedes trabajar con cada 'caja'
171             if verificar_colision(caja_nueva_0, caja):
172                 if(calcular_volumen_caja(caja_nueva_0)==
        calcular_volumen_caja(caja)):
173                     colision1=False

```

```

174         else:
175             colision1 = True
176             break
177     print("colision_en_hijo_1:")
178     print(colision1)
179     #####
180     #####3
181     origen_contenedor0=contenedor0.Center # Contenedor del Grupo 0 (
        asumiendo que es un nico contenedor)
182     # Calcular las dimensiones a partir del BoundingBox
183     contenedor0_BB=contenedor0.ToBrep()
184     contenedor0_BB = contenedor0.ToBrep().GetBoundingBox(True)
185     largo_contenedor0 = contenedor0_BB.Max.X - contenedor0_BB.Min.X
186     ancho_contenedor0 = contenedor0_BB.Max.Y - contenedor0_BB.Min.Y
187     altura_contenedor0 = contenedor0_BB.Max.Z - contenedor0_BB.Min.Z
188
189
190     caja_nueva_1_BB=caja_nueva_1.ToBrep().GetBoundingBox(True)
191     largo_caja_nueva_1 = caja_nueva_1_BB.Max.X - caja_nueva_1_BB.Min.X
192     ancho_caja_nueva_1 = caja_nueva_1_BB.Max.Y - caja_nueva_1_BB.Min.Y
193     alto_caja_nueva_1 = caja_nueva_1_BB.Max.Z - caja_nueva_1_BB.Min.Z
194
195     largo_caja_lista0=[]
196     ancho_caja_lista0=[]
197     altura_caja_lista0=[]
198
199     largo_caja_lista0.append(largo_caja_nueva_1)
200     ancho_caja_lista0.append(ancho_caja_nueva_1)
201     altura_caja_lista0.append(alto_caja_nueva_1)
202
203
204     hijo_0_lista= data_tree_a_lista(hijo_0)
205     #####
206     origen_contenedor1=contenedor1.Center # Contenedor del Grupo 0 (
        asumiendo que es un nico contenedor)
207     # Calcular las dimensiones a partir del BoundingBox
208     contenedor1_BB=contenedor1.ToBrep()
209     contenedor1_BB = contenedor1.ToBrep().GetBoundingBox(True)
210     largo_contenedor1 = contenedor1_BB.Max.X - contenedor1_BB.Min.X
211     ancho_contenedor1 = contenedor1_BB.Max.Y - contenedor1_BB.Min.Y
212     altura_contenedor1 = contenedor1_BB.Max.Z - contenedor1_BB.Min.Z
213
214
215     caja_nueva_0_BB=caja_nueva_0.ToBrep().GetBoundingBox(True)
216     largo_caja_nueva_0 = caja_nueva_0_BB.Max.X - caja_nueva_0_BB.Min.X
217     ancho_caja_nueva_0 = caja_nueva_0_BB.Max.Y - caja_nueva_0_BB.Min.Y
218     alto_caja_nueva_0 = caja_nueva_0_BB.Max.Z - caja_nueva_0_BB.Min.Z
219
220     largo_caja_lista1=[]
221     ancho_caja_lista1=[]

```

```

222 altura_caja_lista1=[]
223
224 largo_caja_lista1.append(largo_caja_nueva_0)
225 ancho_caja_lista1.append(ancho_caja_nueva_0)
226 altura_caja_lista1.append(alto_caja_nueva_0)
227
228
229 hijo_1_lista= data_tree_a_lista(hijo_1)
230
231 #####
232 def generar_contenedor_y_cajas(Cajas,origen_contenedor,
    largo_contenedor, ancho_contenedor, altura_contenedor,
    largo_caja_lista, ancho_caja_lista, altura_caja_lista):
233
234     # Coordinadas origen contenedor
235     contenedor_centro_x = origen_contenedor.X
236     contenedor_centro_y = origen_contenedor.Y
237     contenedor_centro_z = origen_contenedor.Z
238
239     # Funci n para verificar si dos BoundingBoxes se solapan
240     def verificar_colision(caja1, caja2):
241         if(calcular_volumen_caja(caja1)==calcular_volumen_caja(
242             caja2)):
243             return False
244         bbox1 = caja1.ToBrep().GetBoundingBox(True)
245         bbox2 = caja2.ToBrep().GetBoundingBox(True)
246
247         no_colision_x = bbox1.Max.X < bbox2.Min.X or bbox1.Min.X
248             > bbox2.Max.X
249         no_colision_y = bbox1.Max.Y < bbox2.Min.Y or bbox1.Min.Y
250             > bbox2.Max.Y
251         no_colision_z = bbox1.Max.Z < bbox2.Min.Z or bbox1.Min.Z
252             > bbox2.Max.Z
253
254         return not (no_colision_x or no_colision_y or
255             no_colision_z)
256
257     # Funci n para verificar si la caja est completamente
258     dentro del contenedor
259     def verificar_dentro_contenedor(caja, contenedor_centro_x,
260         contenedor_centro_y, contenedor_centro_z,
261         largo_contenedor, ancho_contenedor, altura_contenedor):
262         esquinas = caja.GetCorners()
263
264         for esquina in esquinas:
265             if not (contenedor_centro_x - largo_contenedor / 2
266                 <= esquina.X <= contenedor_centro_x +
267                 largo_contenedor / 2 and
268                 contenedor_centro_y - ancho_contenedor / 2
269                 <= esquina.Y <= contenedor_centro_y +

```

```

259         ancho_contenedor / 2 and
        contenedor_centro_z - altura_contenedor / 2
        <= esquina.Z <= contenedor_centro_z +
        altura_contenedor / 2):
260             return False
261
262     return True
263
264     # Funcion para generar una caja aleatoria sin colisiones
    dentro del contenedor
265     def generar_caja_aleatoria_sin_colision(cajas_existentes,
        contenedor_centro_x, contenedor_centro_y,
        contenedor_centro_z, largo_contenedor, ancho_contenedor,
        altura_contenedor, largo_caja, ancho_caja, altura_caja):
266         intentos = 0
267         max_intentos = 100
268
269         while intentos < max_intentos:
270             x = random.uniform(contenedor_centro_x -
                largo_contenedor / 2 + largo_caja / 2,
                contenedor_centro_x + largo_contenedor / 2 -
                largo_caja / 2)
271             y = random.uniform(contenedor_centro_y -
                ancho_contenedor / 2 + ancho_caja / 2,
                contenedor_centro_y + ancho_contenedor / 2 -
                ancho_caja / 2)
272             z = random.uniform(contenedor_centro_z -
                altura_contenedor / 2 + altura_caja / 2,
                contenedor_centro_z + altura_contenedor / 2 -
                altura_caja / 2)
273
274             punto_origen = rg.Point3d(x, y, z)
275             caja = rg.Box(rg.Plane(punto_origen, rg.Vector3d.
                ZAxis), rg.Interval(0, largo_caja), rg.Interval(
                0, ancho_caja), rg.Interval(0, altura_caja))
276
277             rot_x = random.choice([0, 90, 180, 270])
278             rot_y = random.choice([0, 90, 180, 270])
279             rot_z = random.choice([0, 90, 180, 270])
280
281             if rot_x != 0:
282                 rotacion_x = rg.Transform.Rotation(math.radians(
                    rot_x), rg.Vector3d.XAxis, punto_origen)
283                 caja.Transform(rotacion_x)
284
285             if rot_y != 0:
286                 rotacion_y = rg.Transform.Rotation(math.radians(
                    rot_y), rg.Vector3d.YAxis, punto_origen)
287                 caja.Transform(rotacion_y)
288

```

```

289         if rot_z != 0:
290             rotacion_z = rg.Transform.Rotation(math.radians(
                rot_z), rg.Vector3d.ZAxis, punto_origen)
291             caja.Transform(rotacion_z)
292
293         if not verificar_dentro_contenedor(caja,
            contenedor_centro_x, contenedor_centro_y,
            contenedor_centro_z, largo_contenedor,
            ancho_contenedor, altura_contenedor):
294             intentos += 1
295             continue
296
297         colision = False
298         for caja_existente in cajas_existentes:
299             if verificar_colision(caja, caja_existente):
300                 colision = True
301                 break
302
303         if not colision:
304             return caja
305
306         intentos += 1
307
308         return None
309
310     # Generar el contenedor centrado en el origen
311     contenedor = rg.Box(rg.Plane(origen_contenedor, rg.Vector3d.
        ZAxis),
312                          rg.Interval(-largo_contenedor / 2,
            largo_contenedor / 2),
313                          rg.Interval(-ancho_contenedor / 2,
            ancho_contenedor / 2),
314                          rg.Interval(-altura_contenedor / 2,
            altura_contenedor / 2))
315
316     # Lista para almacenar las cajas generadas
317     #Cajas = []
318
319     # Generar todas las cajas
320     for largo_caja, ancho_caja, altura_caja in zip(
        largo_caja_lista, ancho_caja_lista, altura_caja_lista):
321         caja = generar_caja_aleatoria_sin_colision(Cajas,
            contenedor_centro_x, contenedor_centro_y,
            contenedor_centro_z,
322                                                     largo_contenedor
                ,
                ancho_contenedor
                ,
                altura_contenedor
                ,

```

```

323                                     largo_caja,
324                                     ancho_caja,
325                                     altura_caja)
326
327     if caja is not None:
328         i=0
329         toleranciaa=0.1
330         for caja_veri in Cajas:
331             if abs(calcular_volumen_caja(caja_veri)-
332                   calcular_volumen_caja(caja))<toleranciaa:
333                 Cajas.pop(i)
334                 i=0
335                 break
336             i+=1
337         Cajas.append(caja)
338     else:
339         print(f"No se pudo generar una caja sin colisiones,
340               para las dimensiones: {largo_caja}, {ancho_caja},
341               {altura_caja}")
342
343     # Ajustar las alturas de las cajas
344     #ajustar_altura_cajas(Cajas, altura_contenedor)
345
346     # Retornar las cajas y el contenedor
347     return Cajas, contenedor
348 #####
349 if (colision0):
350     # Generar el contenedor y sus cajas
351     Hijo_0_corregido, contenedor0 = generar_contenedor_y_cajas(
352         hijo_0_lista, origen_contenedor0, largo_contenedor0,
353         ancho_contenedor0, altura_contenedor0, largo_caja_lista0,
354         ancho_caja_lista0, altura_caja_lista0)
355
356 else:
357     Hijo_0_corregido=data_tree_a_lista(hijo_0)
358
359 if (colision1):
360     # Generar el contenedor y sus cajas
361     Hijo_1_corregido, contenedor1 = generar_contenedor_y_cajas(
362         hijo_1_lista, origen_contenedor1, largo_contenedor1,
363         ancho_contenedor1, altura_contenedor1, largo_caja_lista1,
364         ancho_caja_lista1, altura_caja_lista1)
365
366 else:
367     Hijo_1_corregido=data_tree_a_lista(hijo_1)
368
369 Hijos_corregidos.append(Hijo_0_corregido)
370 Hijos_corregidos.append(Hijo_1_corregido)
371
372 return Hijos_corregidos

```

```
362  
363 hijos=cruces_por_pares(cajas_seleccionadas, contenedores_seleccionados)
```